Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · draw.io source in docs/architecture/drawio/

TraceSmith

GPU Profiling & Replay
CUPTI / ROCm / Metal / MACA / Ascend

YiRage YPK

LLM megakernel compiler
12 backends + RuntimeFusion

JAX-sklearn

sklearn API + JAX proxy
CPU / GPU / TPU fallback

Secret-Learn

191 algos × FL / SL / SS
JAX + SecretFlow

YouTube-DNN

Two-tower recall
TensorFlow + Spark

Research CLI

AI CLI + academic tools
multi-provider / MCP

Questions-Gen

Qwen3-14B contest problems
Pretrain → GRPO → Distill

Each following page is a draw.io diagram exported to SVG, then appended to the resume PDF.

TraceSmith — Cross-platform GPU Profiling & Replay

github.com/chenxingqiang/TraceSmith · C++ core + pybind11 · PyPI: tracesmith

1. Workloads / producers

PyTorch / Kineto
kernel + runtime traces

TensorFlow / XLA
graph + device events

hooks

Custom CUDA / HIP
driver / runtime APIs

Multi-GPU cluster
NCCL collectives

2. User interface

CLI
profile / record / view
export / analyze / replay

Python API
pybind11 bindings
in-process capture

Distribution
pip install tracesmith
Apache-2.0

3. Capture backends (platform-specific, lock-free ingest)

NVIDIA CUPTI
nsys / kernel / memcpy

AMD ROCm
rocprof / HIP events

Apple Metal
xctrace / Instruments

MetaX MACA
MCPTI

Huawei Ascend
CANN / msprof

Linux eBPF
host syscall / driver

events

4. Core pipeline

SPSC ring buffer
lock-free ingest
<1% overhead

Common / stacks
event timeline
XRay / host stacks

export

SBT format
string interning
delta timestamps

State / timeline
GPU state machine
stream reconstruction

5. Replay, export, debug

Replay engine
full / partial / dry-run
stream scheduler
determinism checker

Perfetto
protobuf export
~85% smaller than JSON

Tracy profiler
live timeline view

GDB RSP
GPU breakpoints
trace-driven debug

Cluster sync
NTP / PTP / CUDA
NCCL tracking

Flow: workload → CLI/Python → platform backend → lock-free ring → SBT/timeline → replay or Perfetto/Tracy/GDB.

YiRage (YPK) — Multi-backend LLM Megakernel Compiler & Runtime

github.com/chenxingqiang/YiRage · based on Mirage (CMU) + multi-backend / RuntimeFusion extensions

1. Serving / embedding

vLLM
keeps schedule + KV
RF selects capsules

SGLang
decode-loop-erdecode
same FusionPlan cache

Python API
PersistentKernel
HardwareRegistry

UnifiedCompiler
graph + backend pick
Cython CoreBridge

2. RuntimeFusion

FusionPlan cache
search / profile results
per backend + shape

FusionCapsule
one decode-step kernel
selected each token

Device memory
persistent workspace
KV-friendly layout

cache

3. Compiler / search

Kernel graph IR
matmul / attn / norm
mark inputs / outputs

Superoptimize
hardware-aware search
15 candidate dims

Fingerprint
numeric verify
before accept

Autotune
grid / tile / pipeline
occupancy model

4. Threadblock operators + transpilers

MatMul
Tensor Core / Cube

Attention / GQA
Flash-style tiles

RMSNorm / SwiGLU
fused decode path

Triton codegen

NKI / MLIR

CUDA / HIP JIT

5. Backend registry (12 targets; factory + thread-safe singleton)

CUDA
Tensor Core
bank conflict

ROCm
HIP / MFMA

MPS
Apple Metal
unified mem

Ascend
AI Core / Cube
BiSheng

MACA
MetaX

TPU / XPU
NKI / Intel

MLIR / CPU
LLVM / SIMD

FPGA
experimental

Serving keeps scheduling/KV; compiler searches fused kernels; RuntimeFusion injects FusionCapsules into vLLM/SGLang decode.

JAX-sklearn (xlearn) — Drop-in sklearn with automatic JAX acceleration

github.com/chenxingqiang/jax-sklearn · 13,058 tests passed · pip install jax-sklearn

1. User code (zero API change)

import xlearn as sklearn
LinearRegression().fit(X, y)

Same estimators
cluster / decompose / preprocess

config_context()
force GPU / disable JAX

2. sklearn-compatible surface

Linear
LR / Ridge / Lasso
ElasticNet

Cluster
KMeans
282 tests

Decompose
PCA / TruncatedSVD
528 tests

Preprocess
Standard / MinMax

Others
native sklearn
via fallback

3. Runtime injection / proxy

__getattr__ lazy import
xlearn.linear_model …

JAX

_auto_jax_accelerate
replace estimator classes

EstimatorProxy
create JAX or original impl

Heuristics
fallback3 — KMeans > 1e6
PCA > 1e7 complexity

4. Acceleration + data

JAX kernels
JIT / vmap
universal mixins

Algo kernels
GP / MLP / SVM / tree

NumPy ↔ JAX
dtype / shape guards

Fallback
small data or error
→ original sklearn

5. Hardware abstraction (auto-select by problem size)

CPU
<10K samples
1.0–2.5×

NVIDIA GPU
10K–100K
5.5–8×

Google TPU
>100K
9.5–15×

Apple Metal
M1/M2/M3 beta
2–4×

Proxy decides JAX vs sklearn from dataset size, algorithm family, and configured platform; errors fall back silently.

Secret-Learn — Privacy-preserving ML (191 algorithms × FL / SL / SS)

github.com/chenxingqiang/Secret-Learn · sklearn API · JAX acceleration · SecretFlow devices

1. Applications

- **Healthcare** — hospitals keep records local
- **Finance** — cross-bank fraud / risk
- **IoT / edge** — federated sensors
- **Research** — multi-institution studies

2. sklearn-compatible API (191 estimators, 30+ categories)

- Linear / GLM
- Ensemble / trees
- SVM / neighbors
- Cluster / PCA
- NN / Naive Bayes
- Preprocess / select

3. Privacy mode router

- **FL — Federated**
  data stays on PYU
  ~~local JAX train~~
  HEU secure aggregate
  best: horizontal FL
- **SL — Split learning**
  model split by party
  ~~local JAX~~
  encrypted activations
  vertical / deep models
- **SS — Secret sharing**
  SPU MPC encrypt
  maximum privacy
  multi-party runtime

4. Intelligent generation + acceleration

- **Algorithm classifier**
  supervised? partial_fit?
  fit(x) vs fit(x,y)
- **Template migrator**
  auto-generate FL/SL/SS
  adapter from sklearn
- **JAX-sklearn core**
  local 5×+ speedup
  CPU / GPU / TPU
- **573 examples**
  one per algo × mode
  batch runners

5. SecretFlow device plane

- **PYU**
  plaintext local
  party compute
- **HEU**
  homomorphic
  secure aggregate
- **SPU**
  MPC / secret share
- **TEE**
  trusted execution
- **DP hooks**
  optional noise

Same estimator call; mode picks FL/SL/SS. FL keeps data local, SL splits the net, SS runs under MPC on SPU.

YouTube-DNN-RecSys — Two-tower recall (Covington et al.) in TF + Spark

github.com/chenxingqiang/YouTube-DNN-RecSys · Python TensorFlow · Scala Spark · 60 GitHub stars

1. Event / feature sources

Watch history
video / search ids

Context
device / geo / time

Continuous feats
age / counts

Candidate items
catalog embeddings

2. Dual implementation stacks

Python / TensorFlow
data2tfrecords.py
models/dnn.py (Estimator)
custom layers / TensorBoard
SavedModel export

Scala / Spark
FeatureBuilder.scala
BaseSparkLocal
PredictUserVector
ItemEmbeddingPredictor

3. Two-tower model

User tower
item-id embeddings
average watch history
concat continuous feats
MLP → user vector u

Matching
dot(u, v_i)
sampled softmax
next-watch objective

Item tower
shared item embedding
table v_i
optional side features

4. Train / serve

Train
TF Estimator
optional PS / workers
checkpoints

Eval
hold-out next-item
TensorBoard curves

Online recall
ANN / softmax top-k
user vec × item table

Artifacts
TFRecords
SavedModel / Spark vecs

History embeddings are mean-pooled into a user vector and scored against the item table — the paper's candidate-generation stage.

Research CLI — Command-line AI workflow with academic research tools

github.com/iechor-research/research-cli · TypeScript monorepo · Gemini CLI lineage + multi-provider + research modules

1. Interaction

packages/cli
Ink TUI · research.tsx

Slash commands
/research /paper /submit
/model config|select

Non-interactive
npx / CI scripts

VS Code companion
packages/vscode-ide

2. packages/core — orchestration

Prompt + context
GEMINI.md
file discovery

Session state
tool calls ~~compress~~
1M-token window

Policy / sandbox
~~tool permission~~
workspace isolation

Routing
provider + model
fallback chain

3. Multi-model providers

Gemini / Vertex

OpenAI

Anthropic

Baidu ERNIE

DeepSeek

Qwen / Groq / Kimi

4. Tools + MCP

Workspace tools
fs / shell / git

MCP servers
external capabilities

Search grounding
iEchor / web

SDK / a2a-server
programmatic embed

5. Academic research modules (packages/core/src/tools/research)

Bibliography
arXiv / PubMed / IEEE
BibTeX export

Writing
outline / grammar
LaTeX manager

Analysis
stats / citation net
experiment codegen

Submission
journal matcher
template / package

Prompt → core context → selected provider → tools/MCP/research modules → streamed terminal result.

Questions-Gen — Math competition problem generator (Qwen3-14B)

github.com/chenxingqiang/GenQuestion · Unsloth training · Hugging Face: xingqiang/* · pip install questions-gen

1. Training mixture

Historical contests
50% · CUMCM / olympiad

Conditional variants
30% · rewrite constraints

Innovative types
20% · new stems

data_preparer.py
tokenize / pack

2. Three-stage pipeline

Stage 1 — Pretrain
Qwen3-14B SFT
HF: stage1-fp-merged
base generation

Stage 2 — GRPO
8 problems / group
novelty constraint
policy optimization

score

Reward
0.4 difficulty
0.3 novelty
0.2 rigor
0.1 diversity

eval

Stage 3 — Distill
DeepSeek-R1 teacher
difficulty / logic / edu
final merged FP16

3. Validation (questions_gen/validation)

ModelValidator
single-model n tests

BatchValidator
algebra / calculus
geometry / number theory

QualityEvaluator
content / clarity
difficulty / originality

Scores
Final 0.847
teacher 4.2 / 5

4. Package / deploy

CLI
questions-gen
validate / batch
quality / compare

Python API
QuestionsGenTrainer
OllamaManager
HuggingFaceUtils

Hugging Face
stage1 / stage2 / final
xingqiang/*

Ollama
push-all
local generate API

Pretrain → GRPO with multi-dimensional rewards → R1 distillation → validate by category → publish HF weights and Ollama models.